

Grocery App — MVP Plan (Iteration 1)

Oct 7, 2026 · @Voshana

Overview

By Wednesday 14 October 2026, a single-user web app tells me what is at home and what I spend on groceries, using NFC tags and manual entry.

The problem. I shop at Woolworths, Coles or Aldi up to 5 times a week, spending about \$30 to \$60 each visit. I buy items I didn't plan to buy, and I don't track any of it.

What the MVP must prove. Knowing what is at home, and seeing what I spend, reduces unplanned trips and impulse buys.

Success criteria for 14 October:

- Tapping the fridge tag on my iPhone opens the fridge page in under 3 seconds.
- I can log a real shopping trip, with every item and its price, in under 5 minutes.
- Saving a trip updates what the app says is at home.
- The dashboard shows this month's spend, trips this week, and items running low.
- The app is live on the internet, behind a login, and the code is on GitHub.

User stories

Each story becomes one GitHub issue. The format is: *As a [user], I want [action], so that [benefit]*. The acceptance criteria say when the story is done.

#	Story	Done when
1	As the owner, I want to log in, so that only I can see my data.	Login page works; I stay logged in for 30 days; every other page redirects to login if I'm not signed in.
2	As the owner, I want to tap the fridge tag, so that I see what is in the fridge right away.	Tapping a tag opens that location's page, listing items with their status.
3	As the owner, I want to mark an item as plenty, low or out, so that the app matches my fridge.	One tap changes the status without reloading the whole page.

#	Story	Done when
4	As the owner, I want to add a new item to a location, so that things I already had are tracked.	Name, category and location are saved; the item appears on that location's page.
5	As the owner, I want to log a shopping trip with each item's price, so that I know what I spent.	A trip saves its store, date and items; the trip total is correct.
6	As the owner, I want a saved trip to restock my home, so that I don't enter items twice.	Each trip item appears in its chosen location with status plenty.
7	As the owner, I want to see items that are low or out, so that I know what to buy.	The dashboard lists every low or out item, grouped by location.
8	As the owner, I want to see this month's spending by category, so that I know where the money goes.	Monthly total, trip count and a total per category, for any month I pick.
9	As the owner, I want to see past trips, so that I can check what I bought and when.	Trips list, newest first; each opens to show its items and prices.

Scope

Iteration 1 covers the 9 user stories above and nothing else. Anything new goes to the backlog, not into this week.

In scope

- Three locations: Fridge, Pantry, Cleaning cupboard, each with its own NFC tag
- Item status: plenty, low or out
- Manual entry of items and shopping trips (Woolworths, Coles, Aldi, Other)
- Fixed spending categories: fruit & veg, meat & seafood, dairy & eggs, pantry staples, snacks & drinks, toiletries, cleaning, other
- Dashboard: this month's spend, trips this week, low and out items
- Single-user login

Stretch goals (only if the in-scope work is finished by Monday 12 October)

1. Take a product photo with the iPhone camera when adding an item
2. Look up a product's name and picture from Open Food Facts

Backlog (later iterations)

- Exact quantities and units ("3 eggs", "500 g")
- Receipt photo and PDF scanning
- Barcode scanning with calories
- Claude-written monthly spending summary
- Recipes, and checking recipe ingredients against home stock
- Discounts and price history
- Multiple users, external auth (Supabase or Clerk)
- React front end, native app, subscriptions, community recipes, rewards

Tech stack

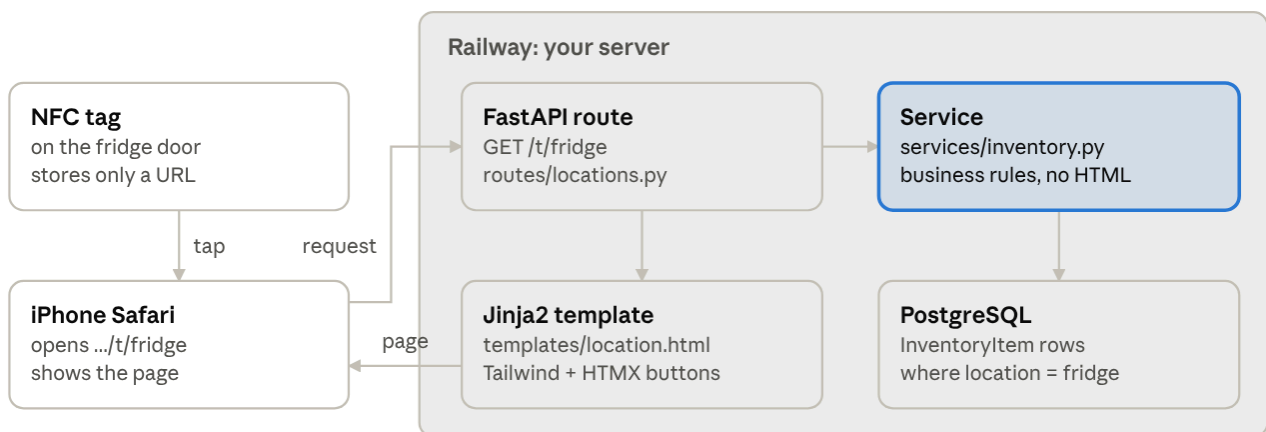
The whole app is Python. The business logic is kept separate from the HTML, so a React front end can replace the pages later without rewriting the backend.

Layer	Choice	Why
Language	Python 3.12	The language you know best
Web framework	FastAPI + Uvicorn	Close to Flask, built for APIs (needed for React later), free <code>/docs</code> test page
Pages	Jinja2 templates	Same templating as Flask; you turn Claude Design's HTML into these
Interactivity	HTMX	Buttons like "mark as low" update part of the page with no JavaScript to write
Styling	Tailwind CSS (CDN)	What Claude Design outputs; no build step needed
Database code	SQLModel	Tables are Python classes; built on SQLAlchemy and Pydantic
Database	SQLite locally, PostgreSQL on Railway	SQLite is just a file; Postgres is what production apps use
Login	passlib (bcrypt) + signed session cookie	Teaches how auth works; enough for one user
Secrets	python-dotenv, <code>.env</code> file	Passwords and keys never go into Git
Tests	pytest + FastAPI TestClient	Tests the rules that matter, like "a trip restocks the fridge"
Code quality	Ruff	Lints and formats on save

Layer	Choice	Why
Hosting	Railway (about US\$5 a month)	App and Postgres in one place, no sleeping, HTTPS for NFC
NFC	NTAG213/215 stickers + NFC Tools app	A tag only stores a URL; the iPhone opens it. No NFC code needed

How a tag tap travels through the system is drawn below.

A tag tap is just a web request to your FastAPI app



tag tap request flow · 6 parts

The highlighted service layer is the part that survives a move to React; only the template layer gets replaced.

Data model

Seven tables. The key idea is that a **product** ("Pauls Full Cream Milk 2L") is kept separate from **what is at home** (an inventory row) and from **what was bought** (a trip item). That way one product can be bought many times and stored in one place.

Table	Fields	Notes
User	id, username, password_hash	One row for now. Never store the plain password.
Location	id, name, slug	Seeded: Fridge (fridge), Pantry (pantry), Cleaning cupboard (cleaning). The slug goes in the tag URL.

Table	Fields	Notes
Category	id, name	Seeded with the 8 fixed categories. A table, not hard-coded, so custom categories are easy later.
Product	id, name, category_id, default_location_id, image_url (optional), created_at	The thing itself. <code>image_url</code> stays empty unless a stretch goal is done.
InventoryItem	id, product_id, location_id, status, quantity (optional), unit (optional), updated_at	What is at home. <code>status</code> is plenty, low or out. <code>quantity</code> and <code>unit</code> stay empty this week; they are there for exact amounts later.
ShoppingTrip	id, store, trip_date, notes, created_at	<code>store</code> is one of Woolworths, Coles, Aldi, Other.
TripItem	id, trip_id, product_id, location_id, price_cents, quantity_bought	One line on the receipt.

Three rules to follow while coding:

1. **Store money as whole cents** (`price_cents = 450` for \$4.50). Decimal numbers in computers round badly, and money must add up exactly.
2. **Don't store the trip total.** Add up the trip items when you need it. A stored total can drift out of sync with its items.
3. **Saving a trip restocks home.** In one database transaction, save the trip and its items, then set each product's inventory row in its location to `plenty`, creating the row if it doesn't exist. Either everything saves or nothing does.

You'll draw these tables and their links in dbdiagram.io during the architecture phase.

Pages and routes

Eight pages plus a few HTMX actions. The three NFC tags point at `/t/fridge`, `/t/pantry` and `/t/cleaning`.

Route	Page or action	Story
GET <code>/login</code> , POST <code>/login</code> , POST <code>/logout</code>	Log in and out	1

Route	Page or action	Story
GET /	Dashboard: month's spend, trips this week, low and out items	7, 8
GET /t/{slug}	Location page (what the tag opens)	2
POST /inventory/{id}/status	HTMX: set plenty, low or out	3
GET /items/new, POST /items	Add an item to a location	4
GET /trips	Past trips, newest first	9
GET /trips/new, POST /trips	Log a trip; HTMX adds item rows to the form	5, 6
GET /trips/{id}	One trip with its items and total	9
GET /spending?month=2026-10	Monthly spend by category	8

Tag URLs once deployed: `https://<your-app>.up.railway.app/t/fridge`. Until the stickers arrive, save these as Safari bookmarks on your iPhone home screen. Tapping one behaves exactly like tapping the tag.

Project structure

The code is split into three layers: **routes** handle web requests, **services** hold the business rules, and **templates** render HTML. Services never touch HTML, which is what makes a later switch to React easy.

```

your-repo/
├─ app/
│   ├─ main.py           # creates the FastAPI app, includes routers
│   ├─ config.py         # reads settings from .env
│   ├─ database.py       # engine and session
│   ├─ models.py         # the 7 SQLAlchemy tables
│   ├─ auth.py           # password hashing, login check
│   ├─ seed.py           # locations, categories, your user
│   ├─ services/         # business rules, no HTML
│   │   ├─ inventory.py
│   │   ├─ trips.py      # save trip + restock home
│   │   └─ spending.py   # monthly totals
│   └─ routes/           # one file per area
│       ├─ auth.py
│       ├─ dashboard.py
│       ├─ locations.py
│       └─ items.py

```

```

| | └─ trips.py
| └─ templates/      # Jinja2, from Claude Design
|   └─ static/       # icons, manifest
└─ tests/
    └─ test_trips.py
    └─ test_inventory.py
    └─ test_spending.py
└─ .env.example      # names of settings, no real values
└─ .gitignore        # includes .env and *.db
└─ requirements.txt
└─ README.md

```

Schedule

20 hours over 8 days, following the engineering lifecycle. Building starts Saturday, so design and architecture are settled before any feature code.

Day	Phase	Hours	What you do	Done when
Wed 7 Oct	Requirements	2	Review this plan; set up GitHub Projects; turn the 9 stories into issues	Board shows 9 issues in To do
Thu 8 Oct	Design	3	Miro: user flows for tap-and-update, log a trip, check spending; rough wireframes; then Claude Design for dashboard, location page and trip form	3 flows and 3 screen designs saved
Fri 9 Oct	Architecture + setup	3	Draw the 7 tables in dbdiagram.io; create venv, install packages, Ruff; FastAPI hello world deployed to Railway; <code>.gitignore</code> , <code>.env.example</code>	<code>/docs</code> loads locally; live on Railway; first commit pushed
Sat 10 Oct	Build	4	Models, database, seed data; location page; status buttons with HTMX; add item	Stories 2, 3, 4 work locally

Day	Phase	Hours	What you do	Done when
Sun 11 Oct	Build	4	Trip form and save with restock; trips list and detail; login	Stories 1, 5, 6, 9 work locally
Mon 12 Oct	Build + test	2	Dashboard and spending page; pytest for the restock rule and monthly totals	Stories 7, 8 work; tests pass
Tue 13 Oct	Deploy	1.5	Railway app + Postgres; environment variables; test on iPhone; bookmarks or tags	Live URL works on iPhone
Wed 14 Oct	Test + review	0.5	Log a real shopping trip; list bugs as issues; short retro	MVP shipped; iteration 2 backlog updated

If you fall behind, cut in this order: the spending page becomes a total on the dashboard, then trip detail is dropped, then login moves to day 1 of iteration 2. Deploying is never cut, because the tags need a live URL.

Ways of working

You write the code that matters; Claude explains, guides and reviews; Claude Code handles boilerplate.

Who writes what

You write	Claude Code may write	Claude (this chat) does
Models, services, routes, tests	Config files, <code>.gitignore</code> , Railway setup, repetitive template markup	Explains each step before you start, reviews commits, answers questions

Git workflow

1. One branch per issue: `feature/3-status-buttons`.
2. Small commits with clear messages: `Add status update endpoint`.
3. Open a pull request into `main`, link the issue (`Closes #3`), then merge. The issue moves to Done on its own.

Setting up GitHub Projects (nothing to install; it's on github.com and in the GitHub mobile app)

1. On your repo page, open the **Projects** tab, then **Link a project → New project**.
2. Pick the **Board** template and name it "MVP — Iteration 1".
3. Keep the columns **Todo**, **In Progress**, **Done**, and add **Backlog** on the left.
4. Create one issue per user story, paste its "Done when" text as a checklist, and add it to the board.
5. Put the backlog items from the Scope section in **Backlog**.

Definition of done for every story: it works on your iPhone, it's merged into `main`, and any business rule it adds has a test.

Risks and open decisions

The biggest risk is time: FastAPI and HTMX are both new to you, and the build days have little slack.

Risk	Effect	Plan
Build takes longer than 10 hours	Stories slip past 14 Oct	Follow the cut order in the Schedule; keep deploy
NFC stickers arrive late	Can't test real taps	Use home-screen bookmarks; tags only store a URL, so nothing changes when they arrive
SQLite and Postgres behave differently	Bugs only appear after deploy	Deploy a hello-world version on Friday, not only on Tuesday
Table changes after go-live (exact quantities)	Real data could be lost	Add Alembic migrations at the start of iteration 2, before changing any table
Secrets pushed to GitHub	Database password exposed	<code>.env</code> in <code>.gitignore</code> from the first commit

Open decisions

- ☐ App name, for the repo, Railway URL and later Reddit posts
- ☐ Repo owner and name, so Claude can attach it and review your code